**Bachelor of Engineering (B.E.) in**
**Computer Engineering (BCT)**
**Electronics, Communication & Information Engineering (BEI)**
**Third Year Second Part**
**Elective II Course**

# Analysis of Algorithms
## *Chapter 6. Backtracking Techniques*


by:

**Er. Bikal Adhikari**

(*B.E., M.Sc. Engg., M.Sc. ISE*)

Assistant Professor, Dept. of Electronics & Computer Engg.

Pulchowk Campus, IOE, TU

# Chapter Outline

## Chapter 6. Backtracking Techniques

4 hours

6.1 Basic concepts
6.2 The N-Queen problem
6.3 Sum of subsets
6.4 Graph coloring
6.5 Hamiltonian cycles
6.6 0/1 knapsack problem using backtracking approach

# Concept of Backtracking

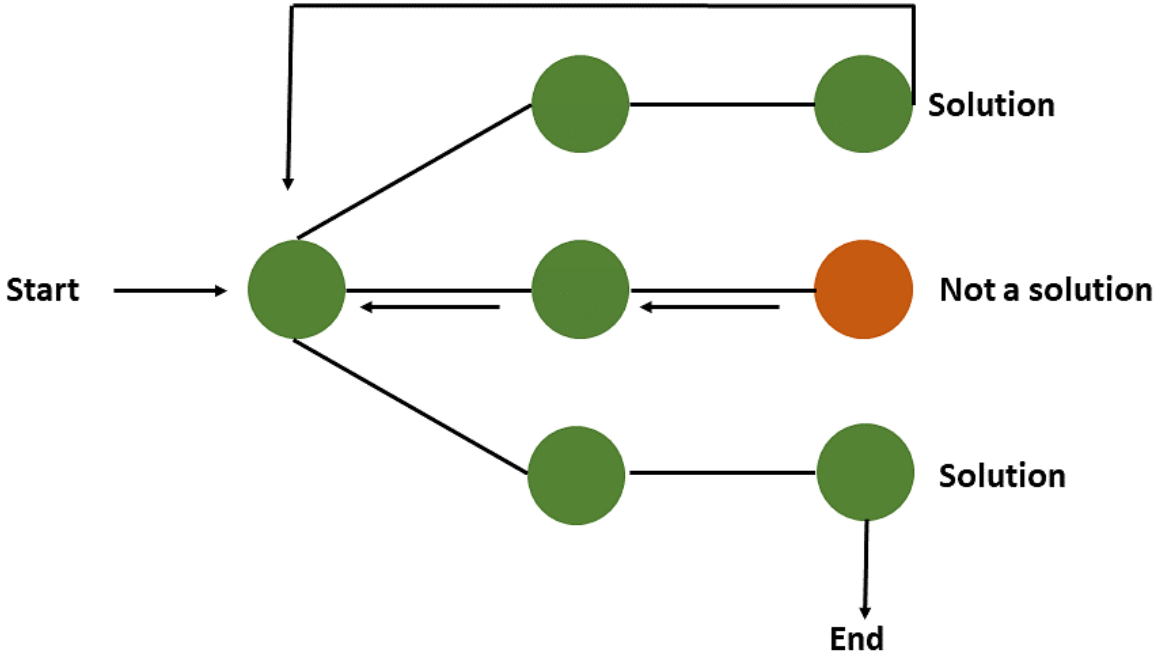
- Backtracking is a methodical way of trying out various sequences of decisions, until you find the correct one that “works”.
- Backtracking is used to solve problems in which a sequence of objects is chosen from a specified set so that the sequence satisfies some criterion.
- Backtracking is a modified depth-first search of a tree.
- It is the procedure whereby, after determining that a node can lead to nothing but dead nodes, we go back (“backtrack”) to the node’s parent and proceed with the search on the next child.

# Recursion Vs Backtracking

- Recursion is a method of solving problems that involves breaking a problem down into smaller and smaller sub-problems until you get to a small enough problem that it can be solved trivially.
- Usually recursion involves a function calling itself.
- While it may not seem like much on the surface, recursion allows to write elegant solutions to problems that may otherwise be very difficult to program.
- In the recursive program, the solution to the base case is provided and the solution to the bigger problem is expressed in terms of smaller problems.
- Example:

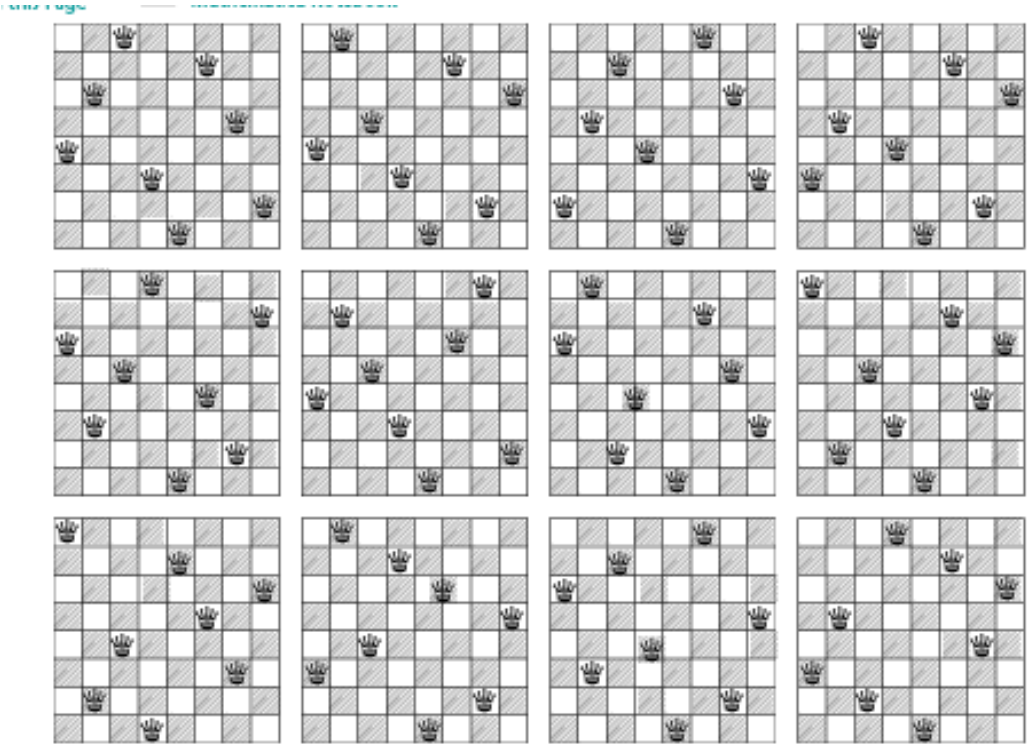
```
int fact(int n)
{
   if(n <=1) //base case
         return 1;
   else
         return n*fact(n-1);
}
```

- Backtracking is an algorithmic technique for solving problems recursively by trying to build a solution incrementally, one piece at a time, removing those solutions that fail to satisfy the constraints of the problem at any point of time.
- Backtracking is a general algorithm for finding solution to the computational problem. We have a set of several choices.
- If one choice from set of choices proves incorrect, computation backtracks or restarts at the point of choice and tries another choice.
- In backtracking, we use recursion in order to explore all possibilities until we get the best result for the problem.



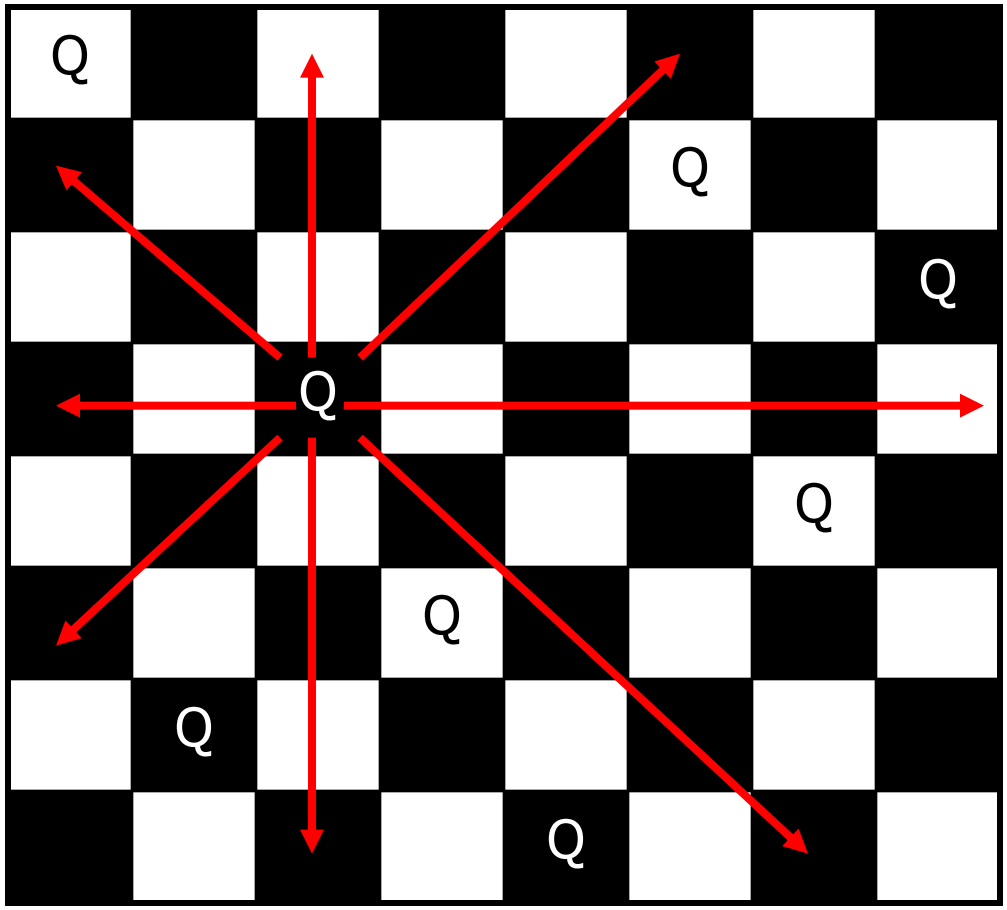
# N Queen Problem

- N Queen Problem: Find a configuration of $N$ queens not attacking each other on a $N \times N$ chessboard.
- What is the maximum number of queens that can be placed on an $N \times N$ chessboard such that no two attack one another?
- The answer is $N$ queens, which gives eight queens for the usual $8 \times 8$ board
- Twelve unique solutions to 8 queen problems:



# The 8 queen problem

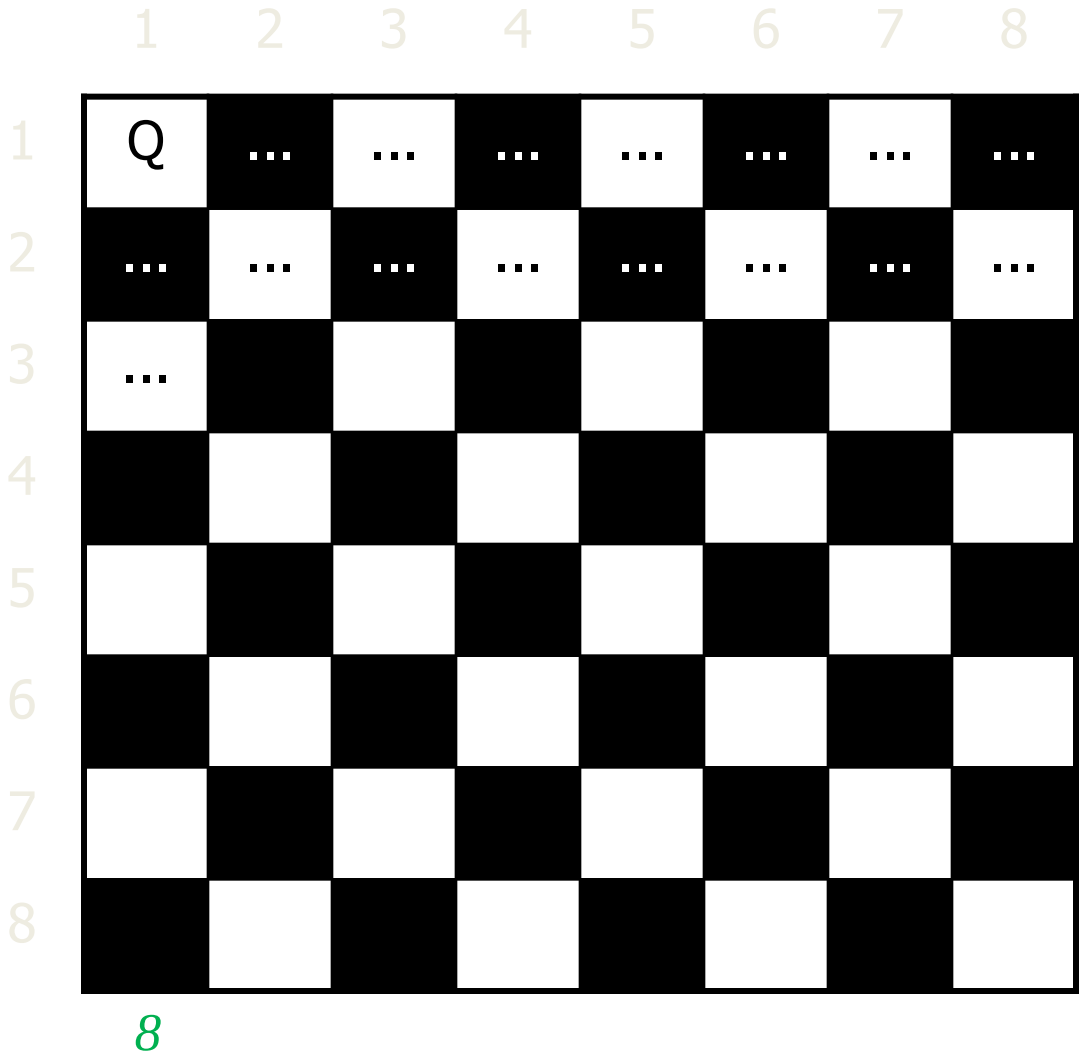
- Consider the problem of trying to place 8 queens on a chess board such that no queen can attack another queen.
- What are the "choices"?
- How do we "make" or "un-make" a choice?
- How do we know when to stop?



# N Queen Problem

**Naive Approach:**

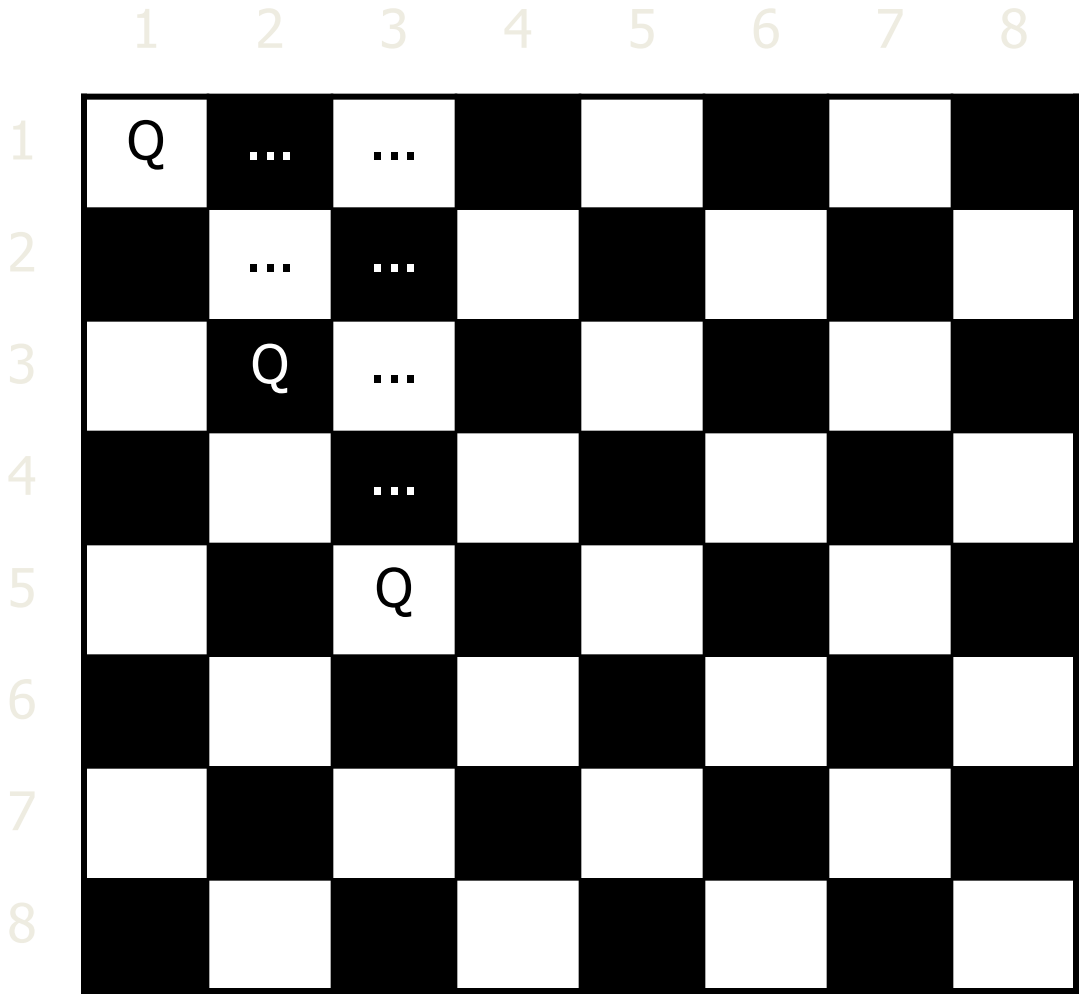
- for (each square on board):
  - Place a queen there.
  - Try to place the rest of the queens.
  - Un-place the queen.
  - How large is the solution space for this algorithm?
    - 64 * 63 * 62 * ...



# N Queen Problem

**Better Algorithm Idea:**

- Observation: In a working solution, exactly 1 queen must appear in each row and in each column.
  - Redefine a "choice" to be valid placement of a queen in a particular column.
  - How large is the solution space now?
    - 8 * 8 * 8 * ...



## Algorithm

Step 1: Start

Step 2: Place the queens column wise in separate rows, start from the left most column

Step 3: If all queens are placed and they aren't attacking each other

3a. Return true and print the solution matrix

Step 4: Else

4a. Try all the rows in the current column

4b. Check if queen can be placed here safely, if yes mark the current cell in the solution

4c. If placing the queen in the above step leads to the solution return true

4d. If placing the queen in the above step does not lead to the solution, backtrack, mark the current cell in the solution matrix as 0 and return false.

Step 5: If all the rows are tried and nothing worked, then return false and print no solution.

Step 6: Stop

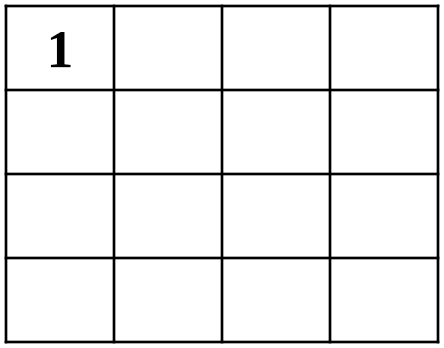
**Analysis:**

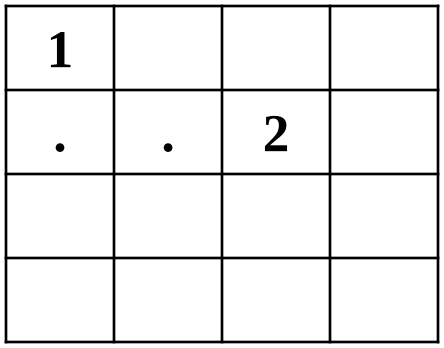
Solution of N queen problem using backtracking checks for all possible arrangements of N queens on the chess board.

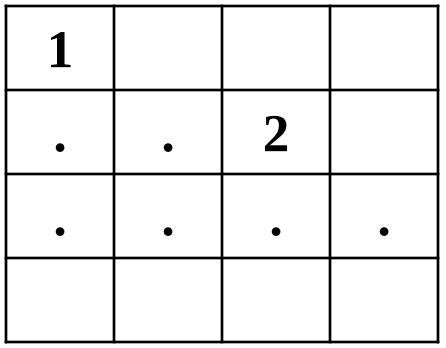
And then checks for the validity of the solution. Now the number of possible arrangements of N queens on N×N chessboard is N!. So average and worst case complexity of the solution is O(N!). The base case occurs if we find our solution before exploiting all possible arrangements. This depends upon our implementation. And if we need all the possible solutions, the best, average and worst case complexity remains O(N!).

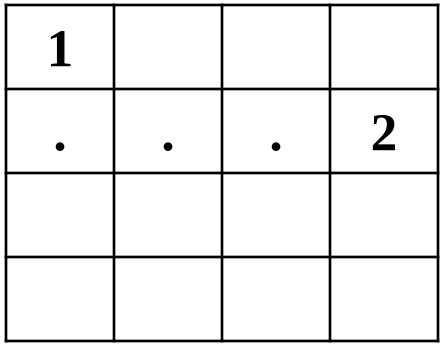
## Example

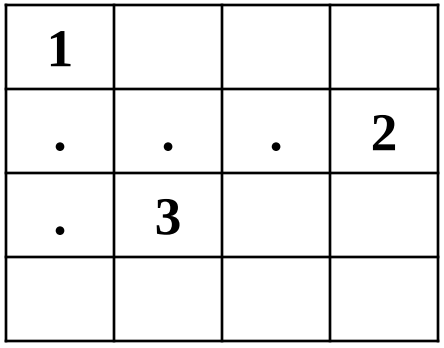
Solution of N Queen Problem for N = 4

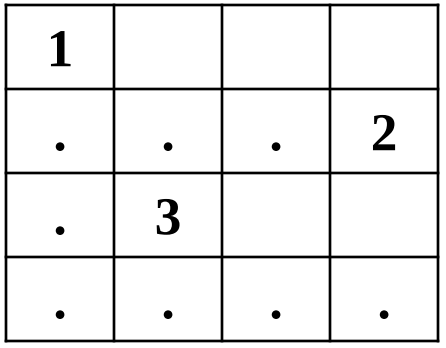


## Contd…

| | 1 | | |
|---|---|---|---|
| | | | |
| | | | |
| | | | |

| | 1 | | |
|---|---|---|---|
| . | . | . | 2 |
| | | | |
| | | | |

| | 1 | | |
|---|---|---|---|
| . | . | . | 2 |
| 3 | | | |
| | | | |

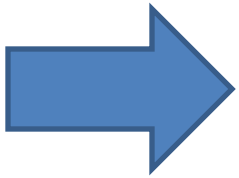
| | 1 | | |
|---|---|---|---|
| . | . | . | 2 |
| 3 | | | |
| . | . | 4 | |

Which is one of the solutions of 4-Queen problem.
The sequence is:
{2,4,1,3}

By mirroring the above solution we can get another possible solution.

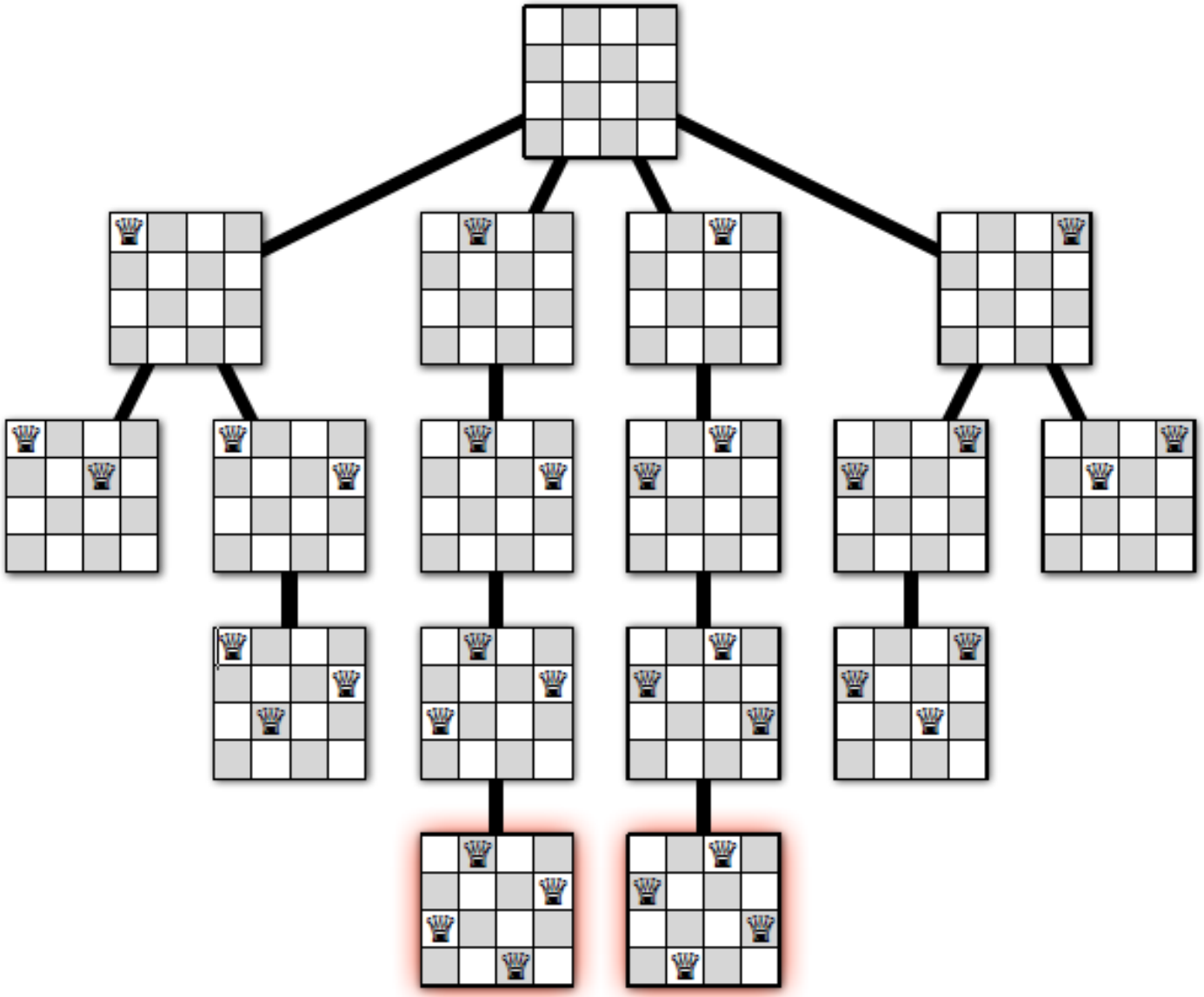
| | | | |
|---|---|---|---|
| | **1** | | |
| **.** | **.** | **.** | **2** |
| **3** | | | |
| **.** | **.** | **4** | |

Mirroring



| | | | |
|---|---|---|---|
| | | **1** | |
| **2** | **.** | **.** | **.** |
| **.** | **.** | **.** | **3** |
| **.** | **4** | **.** | |

The sequence is:
{3,1,4,2}



The complete recursion tree for our algorithm for the 4 queens problem.

# Subset Sum Problem

- In this problem, a set is given with some integer elements and a value is provided.
- The problem is to find a subset of the given set whose sum of elements is equal to the given value.
- In this problem backtracking algorithm is used for trying to select a valid subset when an item is not valid, we will backtrack to get the previous subset and add another element to get the solution.
- The subset-sum problem is to find a subset S′ of the given set $S = \{S1, S2, S3, \ldots, Sn\}$ where the elements of the set $S$ are $n$ positive integers in such a manner that S′ belongs to S and the sum of elements of subset $S'$ is equal to some positive integer $X$.
- The subset-sum problem can be solved by using the backtracking approach. In this implicit tree is a binary tree. The root of the tree is selected in such a way that represents that no decision is yet taken on any input. We assume that the elements of the given set are in the increasing order:
- $S1 \leq S2 \leq S3 \leq \ldots. \leq Sn$
- The left child of the root node indicate that we have to include S1 from the set S and right child of the root indicates we have to exclude S1. Each node stores the total of the partial solution elements. If at any stage sum equals to X then search is successful and terminates.

- The dead end in the tree appears only when either of the two inequalities exists:
  - The sum of the S' is too large
  - The sum of S' is too small

**Algorithm:**

Step 1: Start

Step 2: if index == array.length then

return FALSE

Step 3: if array[index]==sum then

return TRUE

Step 4: Iterate given array from index to array.length

if array[i] > sum then

Don’t do anything take next element from array

if array[i] == sum then

return TRUE

Recursively call with index +1 and sum-array[i]

if last recursive call was success then

return TRUE

Step 5: return FALSE

Step 6: Stop

# Example

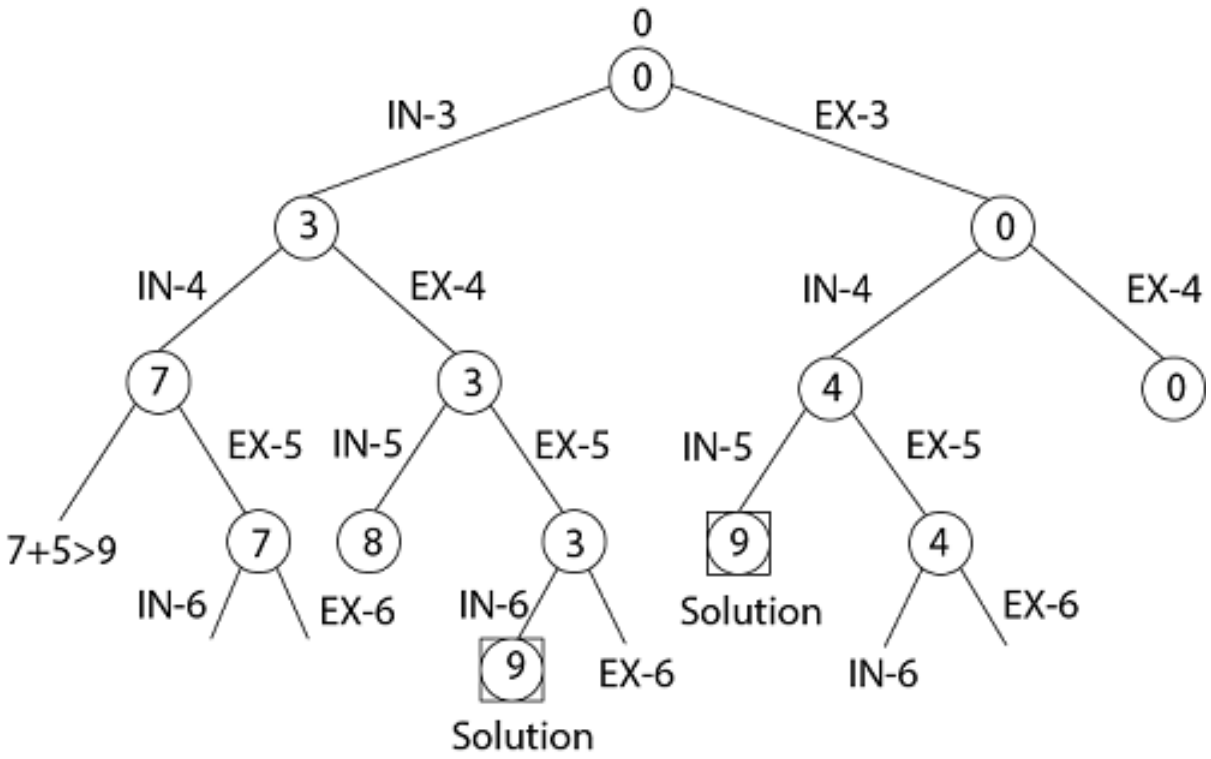
***Given a set S = {3, 4, 5, 6} and X = 9, obtain the subset sum using backtracking approach.***

***Solution:***

Initially S = {3,4,5,6} and X = 9

S′ = Φ

The implicit binary tree for the subset sum problem is shown in the following figure:



# Time Complexity

Time Complexity is $O(2^n)$

# Graph Coloring

→ Graph coloring is nothing but a simple way of labeling graph components such as vertices, edges, and regions under some constraints.

→ In a graph, no two adjacent vertices, adjacent edges, or adjacent regions are colored with minimum number of colors.

→ This number is called the chromatic number and the graph is called a properly colored graph.

→ While graph coloring, the constraints that are set on the graph are colors, order of coloring, the way of assigning color, etc.

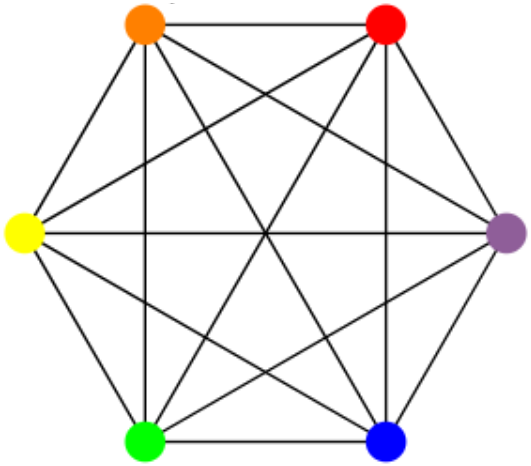
→ A coloring is given to a vertex or a particular region. Thus, the vertices or regions having same colors form independent sets.

→ ***Vertex Coloring*:** Vertex coloring is an assignment of colors to the vertices of a graph 'G' such that no two adjacent vertices have the same color. Simply put, no two vertices of an edge should be of the same color.

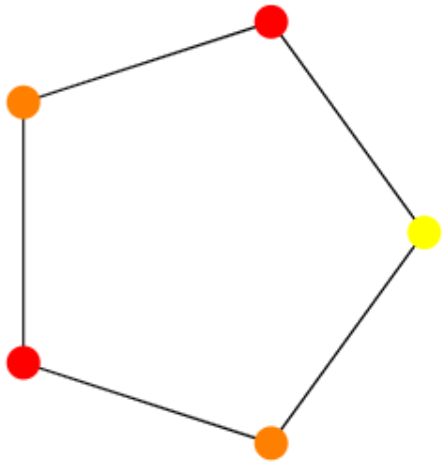
→ ***Chromatic Number***: The minimum number of colors required for vertex coloring of graph 'G' is called as the chromatic number of G, denoted by $\chi$ (G). $\chi(G) = 1$ if and only if 'G' is a null graph. If 'G' is not a null graph, then $\chi(G) \geq 2$.

# Graph Coloring

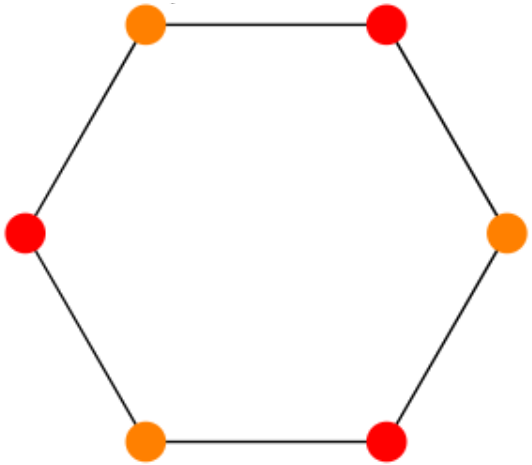
$\chi(K_6) = 6$



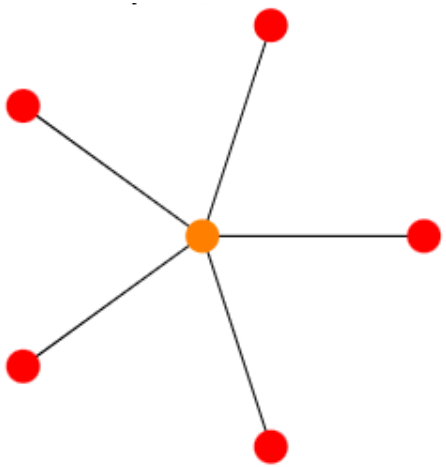
$\chi(C_5) = 3$



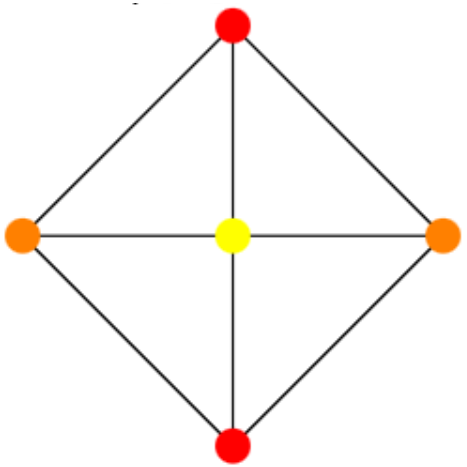
$\chi(C_6) = 2$



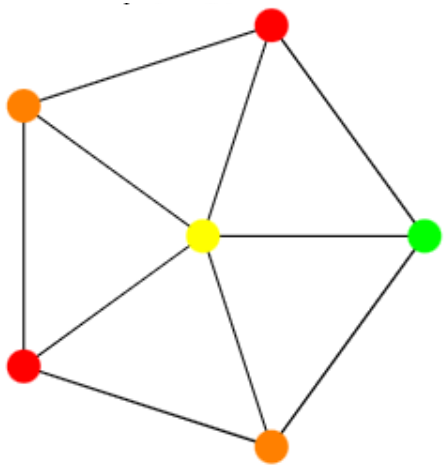
$\chi(S_6) = 2$



$\chi(W_5) = 3$



$\chi(W_6) = 4$



# Graph Coloring

→ Let $G$ be a graph and $m$ be a given positive integer. We want to discover whether the nodes of $G$ can be colored in such a way that no two adjacent nodes have the same color yet only $m$ colors are used.

→ This is termed the $m$-colorability decision problem

→ Note that if $d$ is the degree of the given graph, then it can be colored with $d + 1$ colors. The $m$-colorability optimization problem asks for the smallest integer $m$ for which the graph $G$ can be colored.

→ This integer is referred to as the chromatic number of the graph. For example, the graph of following figure can be colored with three colors 1, 2, and 3. The color of each node is indicated next to it. It can also be seen that three colors are needed to color this graph and hence this graph's chromatic number is 3.

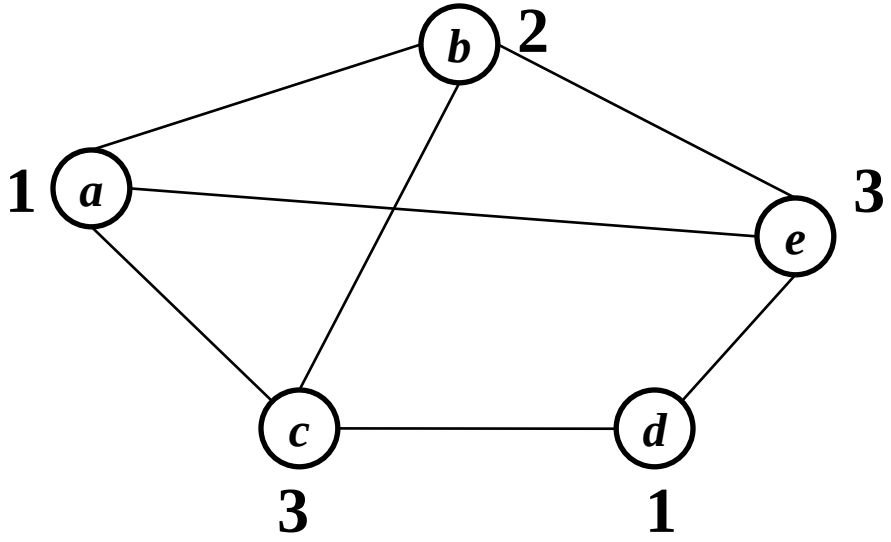



Figure: *An example graph and its coloring*

# Graph Coloring

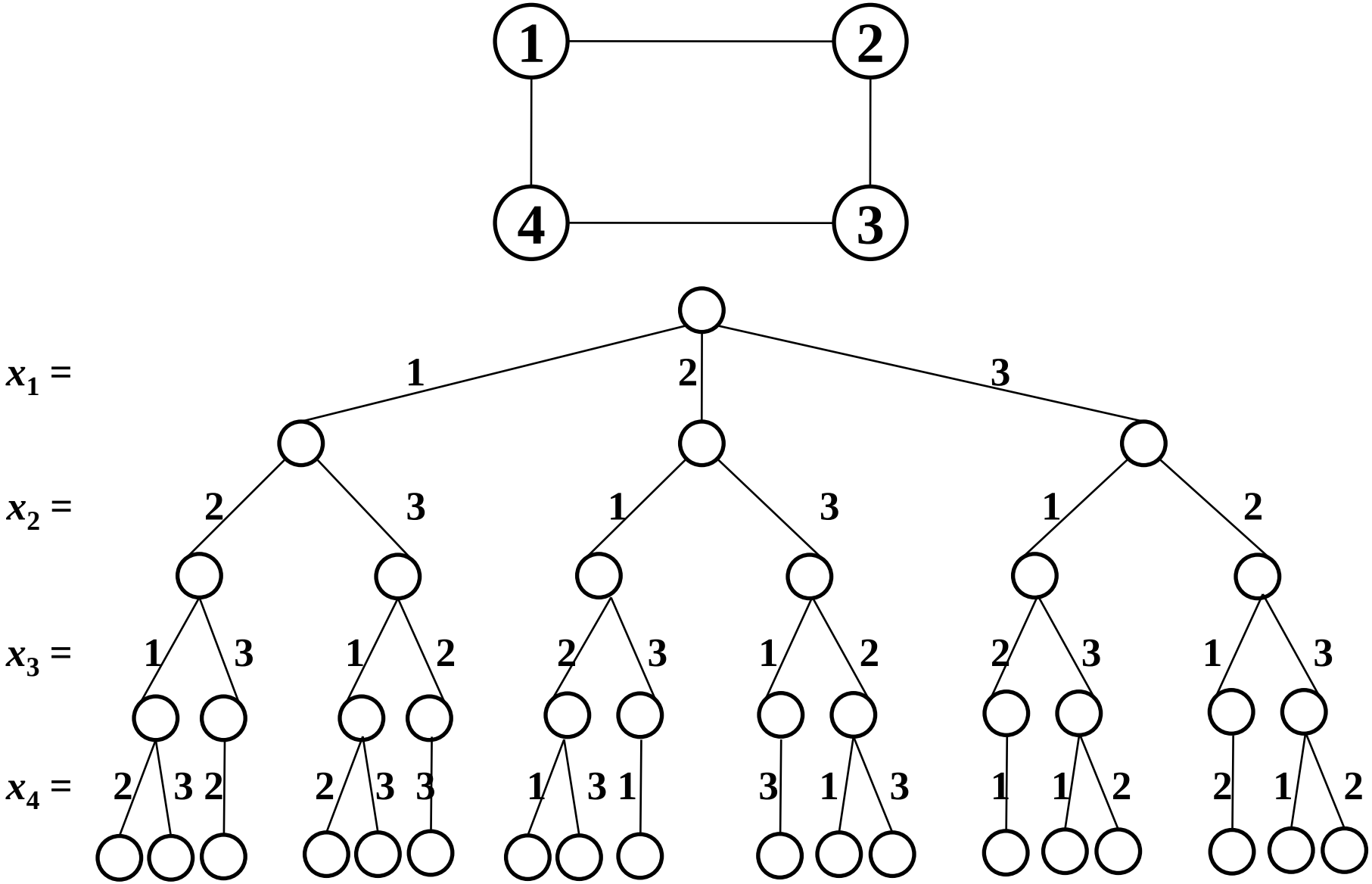
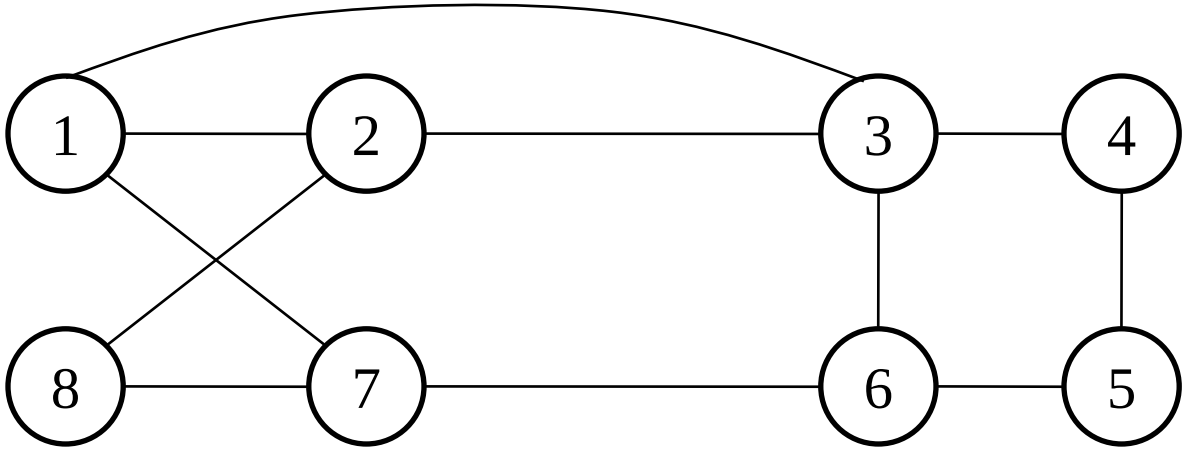



Figure: *A* 4 *node graph and all possible* 3 *colorings*

# Hamiltonian Cycle

G1:




G2:

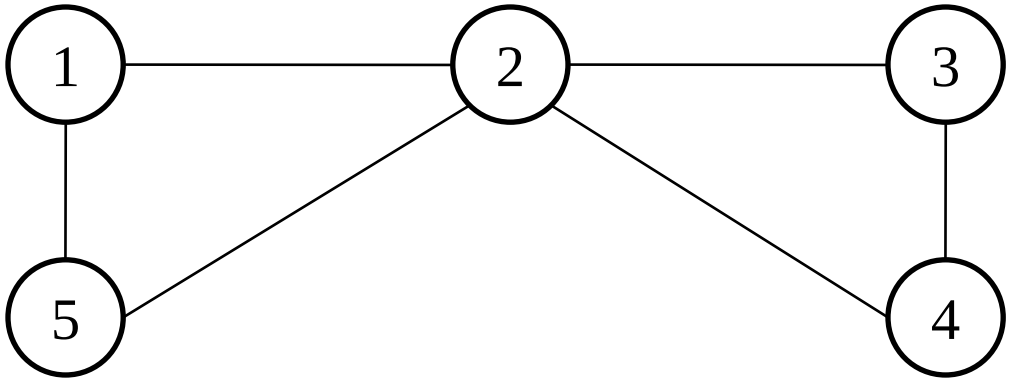



Figure: *Two graphs, one containing a Hamiltonian cycle*

# Hamiltonian Cycle

- → Let ($G = (V, E)$) be a connected graph with ($n$) vertices.
- → A Hamiltonian cycle (suggested by Sir William Hamilton) is a round-trip path along ($n$) edges of ($G$) that visits every vertex once and returns to its starting position.
- → In other words if a Hamiltonian cycle begins at some vertex ($v_1$ in $G$) and the vertices of (G) are visited in the order ($v_1, v_2, \ldots, v_{n+1}$), then the edges (($v_i, v_{i+1}$)) are in ($E$), ($1 \leq i \leq n$), and the ($v_i$) are distinct except for ($v_1$) and ($v_{n+1}$), which are equal.
- → The graph G1 of the figure contains the Hamiltonian cycle (1, 2, 8, 7, 6, 5, 4, 3, 1).
- → The graph G2 of the figure contains no Hamiltonian cycle.
- → There is no known easy way to determine whether a given graph contains a Hamiltonian cycle.
- → We now look at a backtracking algorithm that finds all the Hamiltonian cycles in a graph.
- → The graph may be directed or undirected. Only distinct cycles are output.

# Hamiltonian Cycle

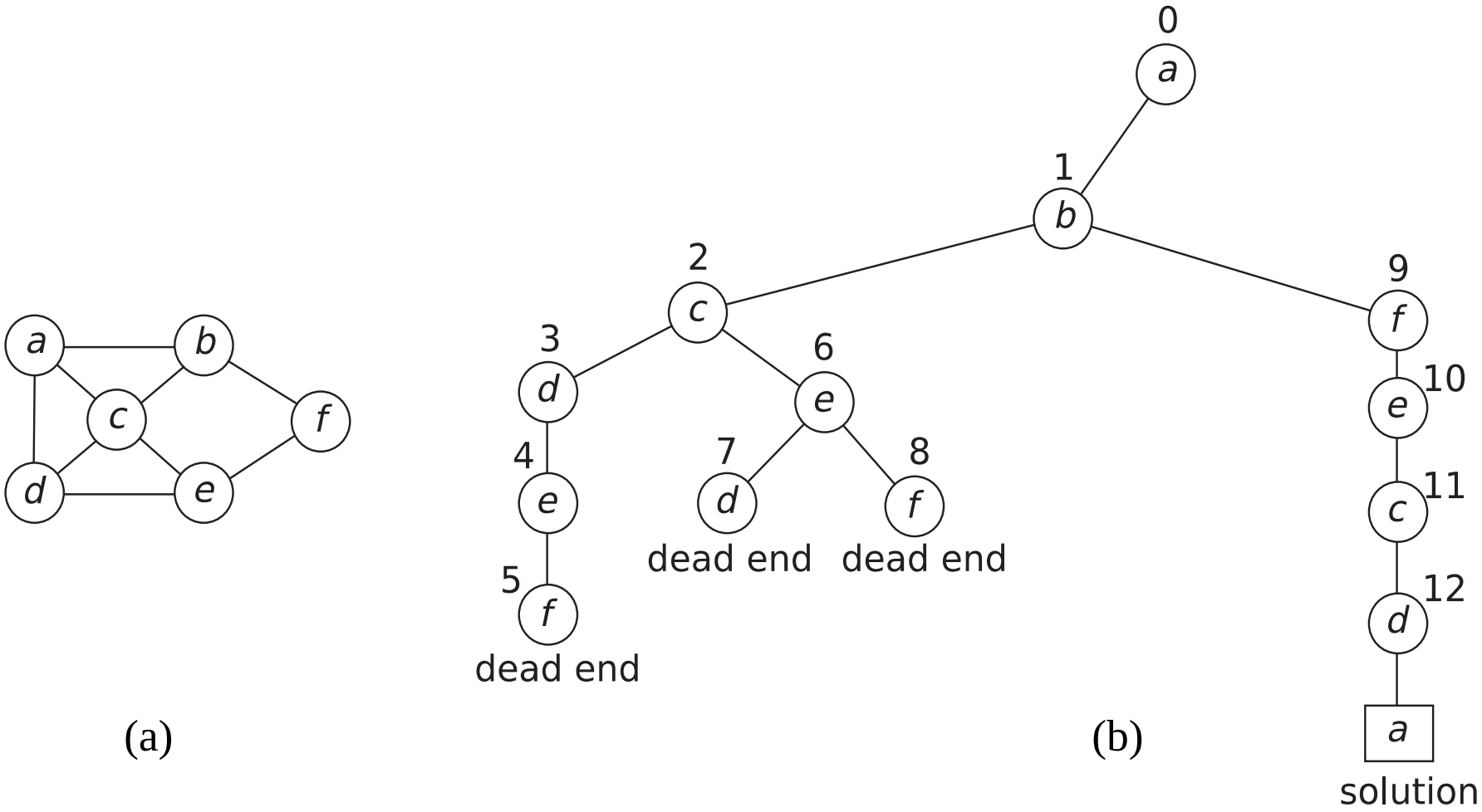



Figure: (a) Graph. (b) State-space tree for finding a Hamiltonian circuit. The numbers above the nodes of the tree indicate the order in which the nodes are generated.

**Source: (Levitin, 2012)**

# Hamiltonian Cycle

Without loss of generality, we can assume that if a Hamiltonian circuit exists, it starts at vertex *a*. *Accordingly, we make vertex a the root of the state-space* tree (Figure). The first component of our future solution, if it exists, is a first intermediate vertex of a Hamiltonian circuit to be constructed. Using the alphabet order to break the three-way tie among the vertices adjacent to *a*, *we* select vertex *b*. *From b, the algorithm proceeds to c, then to d, then to e, and* finally to *f, which proves to be a dead end. So the algorithm backtracks from f* to *e, then to d, and then to c, which provides the first alternative for the algorithm* to pursue. Going from *c to e eventually proves useless, and the algorithm has to* backtrack from *e to c and then to b. From there, it goes to the vertices f , e, c, and d, from which it can legitimately return to a, yielding the Hamiltonian circuit a, b, f , e, c, d, a. If we wanted to find another Hamiltonian circuit, we could continue* this process by backtracking from the leaf of the solution found.

# Zero-One(0/1) Knapsack Problem

- Given a set of items, each with a weight and a value, determine the number of each item to include in a collection so that the total weight is less than or equal to the given limit and the total value is as large as possible.
- It derives its name from the problem faced by someone who is constrained by a fixed-size knapsack and must fill it with the most valuable items.
- The problem often arises in resource allocation where there are financial constraints and is studied in the fields such as combinatorics, computer science, complexity theory, cryptography, applied mathematics and daily fantasy sports.
- **Statement:** A thief has a bag or knapsack that can contain maximum weight *W* of his loot. There are n items and the weight of $i^{th}$ item is $w_i$ and its worth is $v_i$. An amount of item can bet put into the bag is 0 or 1. i.e. $x_i$ is 0 or 1. Here, the objective is to collect the items that maximize the total profit earned.

# Example

**Find maximum profit earned by using 0/1 knapsack problem of Backtracking for the following data items with their respective weights and values.**

Items = {1, 2, 3, 4}

Weight= {2, 3, 4, 5}

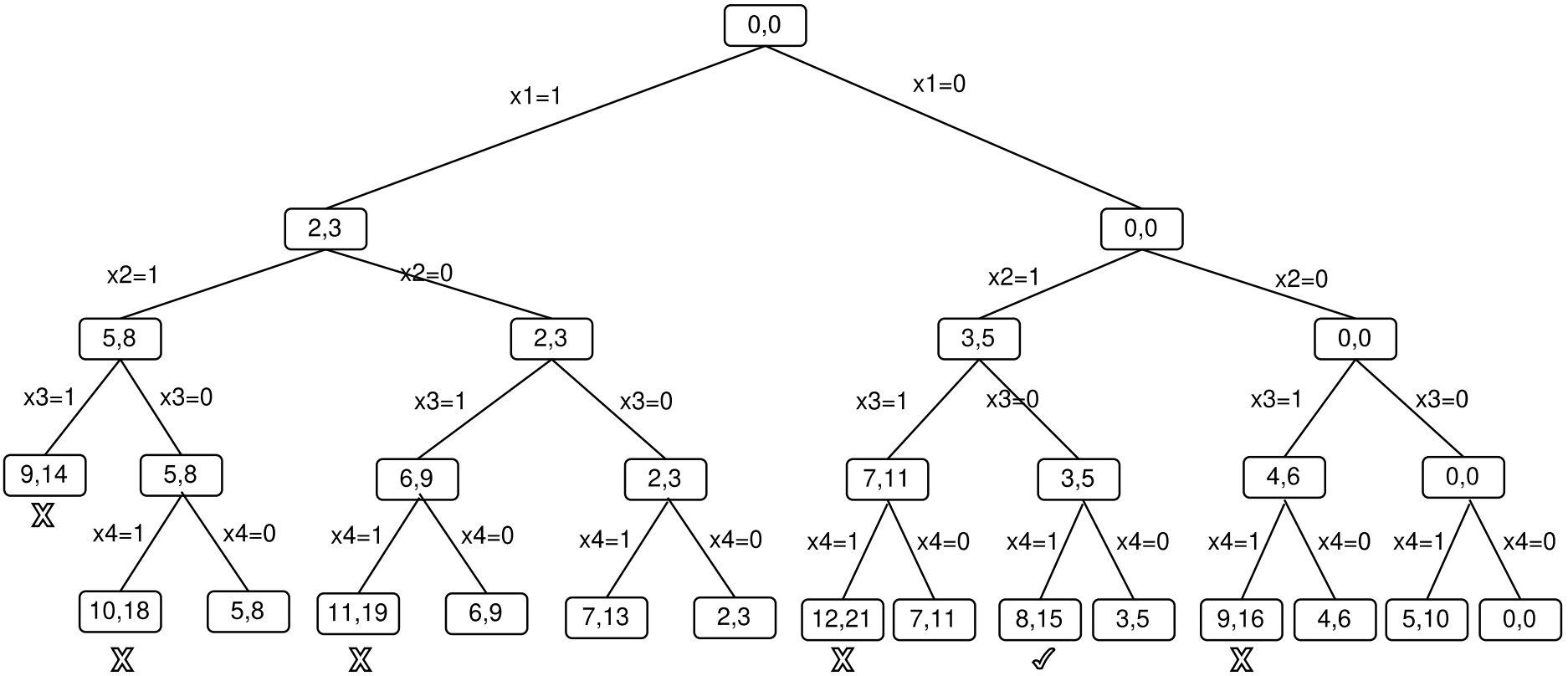
Values = {3, 5, 6, 10}

And knapsack of weight, $W = 8$

**Solution:**

**P.T.O**

# Example


0,0
x1=1
x1=0
2,3
0,0
x2=1
x2=0
x2=1
x2=0
5,8
2,3
3,5
0,0
x3=1
x3=0
x3=1
x3=0
x3=1
x3=0
x3=1
x3=0
9,14
5,8
6,9
2,3
7,11
3,5
4,6
0,0
x4=1
x4=0
x4=1
x4=0
x4=1
x4=0
x4=1
x4=0
x4=1
x4=0
x4=1
x4=0
10,18
5,8
11,19
6,9
7,13
2,3
12,21
7,11
8,15
3,5
9,16
4,6
5,10
0,0

# Time Complexity

Time Complexity is $O(2^n)$

*Thank You!*

*Any Queries?*